

AI ASSISTED CODING

ASSIGNMENT-6.3

B.GEETHANJALI

H.T.NO: 2303A51005

Task Description #1 (Loops – Automorphic Numbers in a Range)

- Task: Prompt AI to generate a function that displays all Automorphic numbers between 1 and 1000 using a for loop.

- Instructions:

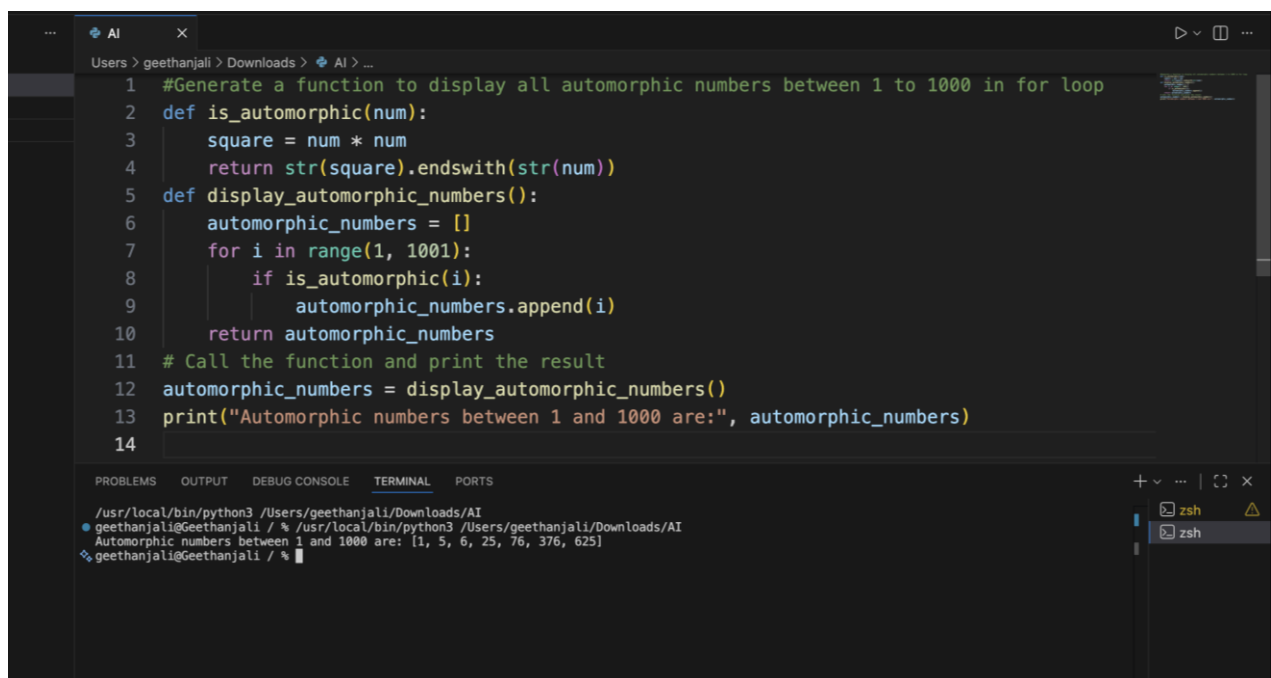
o Get AI-generated code to list Automorphic numbers using a for loop. o

Analyze the correctness and efficiency of the generated logic. o Ask AI to

regenerate using a while loop and compare both implementations.

Expected Output #1:

- Correct implementation that lists Automorphic numbers using both loop types, with explanation.

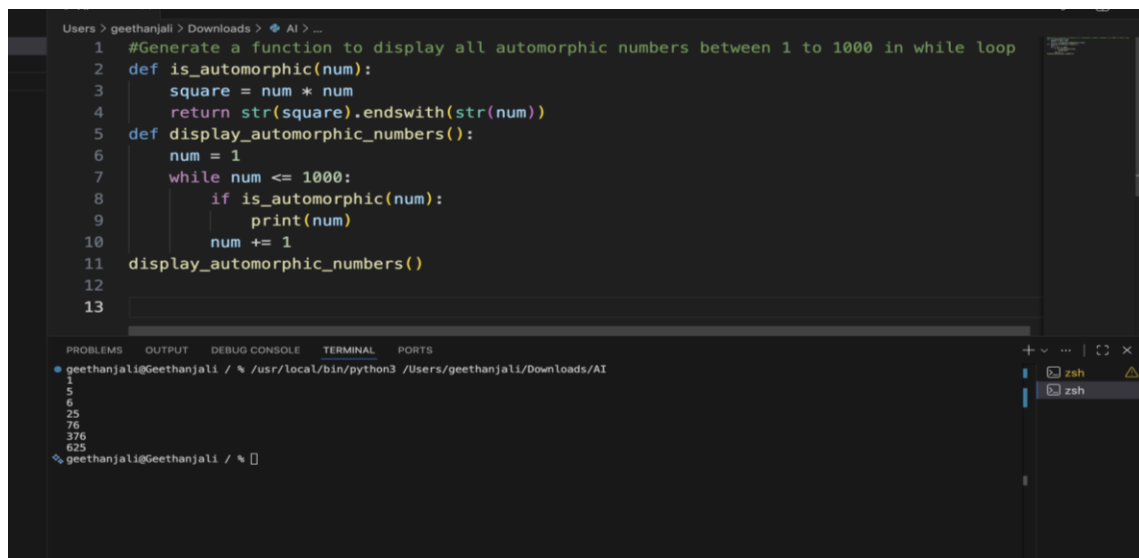


The screenshot shows a code editor with a Python script to find automorphic numbers. The code defines a function `is_automorphic` that checks if a number's square ends with the number itself. It then uses a `for` loop to find all such numbers between 1 and 1000. The output of the script is displayed in the terminal window at the bottom.

```
1 #Generate a function to display all automorphic numbers between 1 to 1000 in for loop
2 def is_automorphic(num):
3     square = num * num
4     return str(square).endswith(str(num))
5 def display_automorphic_numbers():
6     automorphic_numbers = []
7     for i in range(1, 1001):
8         if is_automorphic(i):
9             automorphic_numbers.append(i)
10    return automorphic_numbers
11 # Call the function and print the result
12 automorphic_numbers = display_automorphic_numbers()
13 print("Automorphic numbers between 1 and 1000 are:", automorphic_numbers)
14
```

Terminal Output:

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Automorphic numbers between 1 and 1000 are: [1, 5, 6, 25, 76, 376, 625]
geethanjali@Geethanjali / %
```



```
1 #Generate a function to display all automorphic numbers between 1 to 1000 in while loop
2 def is_automorphic(num):
3     square = num * num
4     return str(square).endswith(str(num))
5 def display_automorphic_numbers():
6     num = 1
7     while num <= 1000:
8         if is_automorphic(num):
9             print(num)
10            num += 1
11 display_automorphic_numbers()
12
13
```

Terminal Output:

```
1
5
6
25
76
376
625
```

1. Both methods take about the same time because each number is checked once and string conversion adds a little extra work, so time complexity is $O(n \log n)$.
2. They use very little memory since only a small list of automorphic numbers is stored.
3. A for loop is faster because it is optimized inside Python.
4. A for loop makes the code cleaner and easier to understand because no manual counter is needed.
5. A for loop is safer and less error-prone since it avoids skipping numbers or running into infinite loops.

Task Description #2 (Conditional Statements – Online Shopping Feedback Classification)

- Task: Ask AI to write nested if-elif-else conditions to classify online shopping feedback as Positive, Neutral, or Negative based on a numerical rating (1–5).
- Instructions:
 - o Generate initial code using nested if-elif-else.
 - o Analyze correctness and readability.
 - o Ask AI to rewrite using dictionary-based or match-case structure.

Expected Output #2:

- Feedback classification function with explanation and an alternative approach.

```
Users > geethanjali > Downloads > AI > ...
1 #Generate a nested if_elif_else to classify shopping feedback negative, neutral, or positive
2 rating = int(input("Please enter your shopping feedback rating (1-10): "))
3 if rating >= 1 and rating <= 10:
4     if rating >= 1 and rating <= 4:
5         print("Your feedback is classified as: Negative")
6     elif rating >= 5 and rating <= 7:
7         print("Your feedback is classified as: Neutral")
8     else:
9         print("Your feedback is classified as: Positive")
10 else:
11     print("Invalid rating. Please enter a number between 1 and 10.")
12
13
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Please enter your shopping feedback rating (1-10): 8
Your feedback is classified as: Positive
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Please enter your shopping feedback rating (1-10): 5
Your feedback is classified as: Neutral
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Please enter your shopping feedback rating (1-10): 2
Your feedback is classified as: Negative
geethanjali@Geethanjali / %
```

```
Users > geethanjali > Downloads > AI > ...
1 #Generate code to classify shopping feedback negative, neutral, or positive based on the r
2 def classify_feedback(rating):
3     feedback_dict = {
4         range(1, 4): "negative",
5         range(4, 7): "neutral",
6         range(7, 11): "positive"
7     }
8
9     for rating_range, feedback in feedback_dict.items():
10         if rating in rating_range:
11             return feedback
12
13     return "Invalid rating"
14 # Example usage
15 rating = 8
16 feedback = classify_feedback(rating)
17 print(f"The feedback for rating {rating} is: {feedback}")
18
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
The feedback for rating 8 is: positive
geethanjali@Geethanjali / %
```

1. A dictionary makes things clear because all rating-to-feedback pairs are visible in one place.
2. It is easy to maintain since you can add or change ratings without changing the main logic.
3. It handles invalid ratings more safely by letting you check or give a default message.
4. It follows Python's standard and clean way of mapping values, so the code looks natural.
5. It stays clean and readable even if the number of rating categories increases.

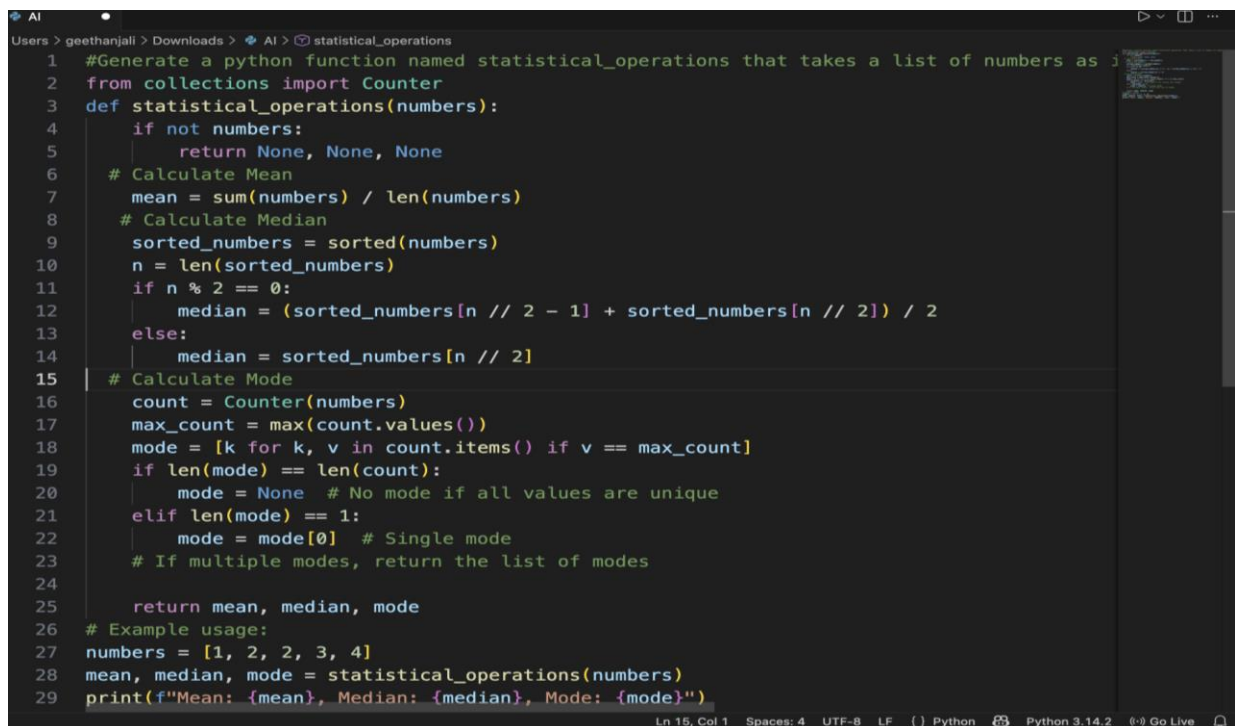
Task 3: Statistical_operations

Define a function named `statistical_operations(tuple_num)` that performs the following statistical operations on a tuple of numbers:

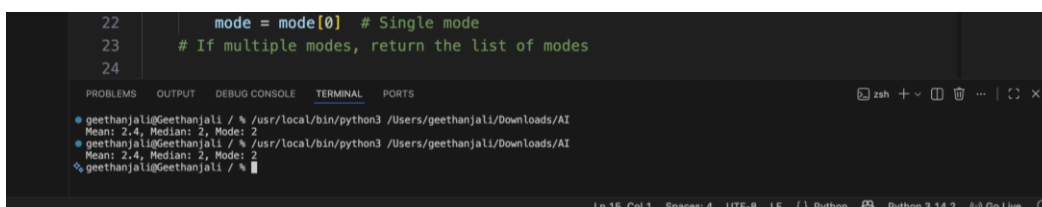
- Minimum, Maximum
- Mean, Median, Mode
- Variance, Standard Deviation

While writing the function, observe the code suggestions provided by GitHub

Copilot. Make decisions to accept, reject, or modify the suggestions based on their relevance and correctness



```
1 #Generate a python function named statistical_operations that takes a list of numbers as input
2 from collections import Counter
3 def statistical_operations(numbers):
4     if not numbers:
5         return None, None, None
6     # Calculate Mean
7     mean = sum(numbers) / len(numbers)
8     # Calculate Median
9     sorted_numbers = sorted(numbers)
10    n = len(sorted_numbers)
11    if n % 2 == 0:
12        median = (sorted_numbers[n // 2 - 1] + sorted_numbers[n // 2]) / 2
13    else:
14        median = sorted_numbers[n // 2]
15    # Calculate Mode
16    count = Counter(numbers)
17    max_count = max(count.values())
18    mode = [k for k, v in count.items() if v == max_count]
19    if len(mode) == len(count):
20        mode = None # No mode if all values are unique
21    elif len(mode) == 1:
22        mode = mode[0] # Single mode
23    # If multiple modes, return the list of modes
24
25    return mean, median, mode
26 # Example usage:
27 numbers = [1, 2, 2, 3, 4]
28 mean, median, mode = statistical_operations(numbers)
29 print(f"Mean: {mean}, Median: {median}, Mode: {mode}")
```

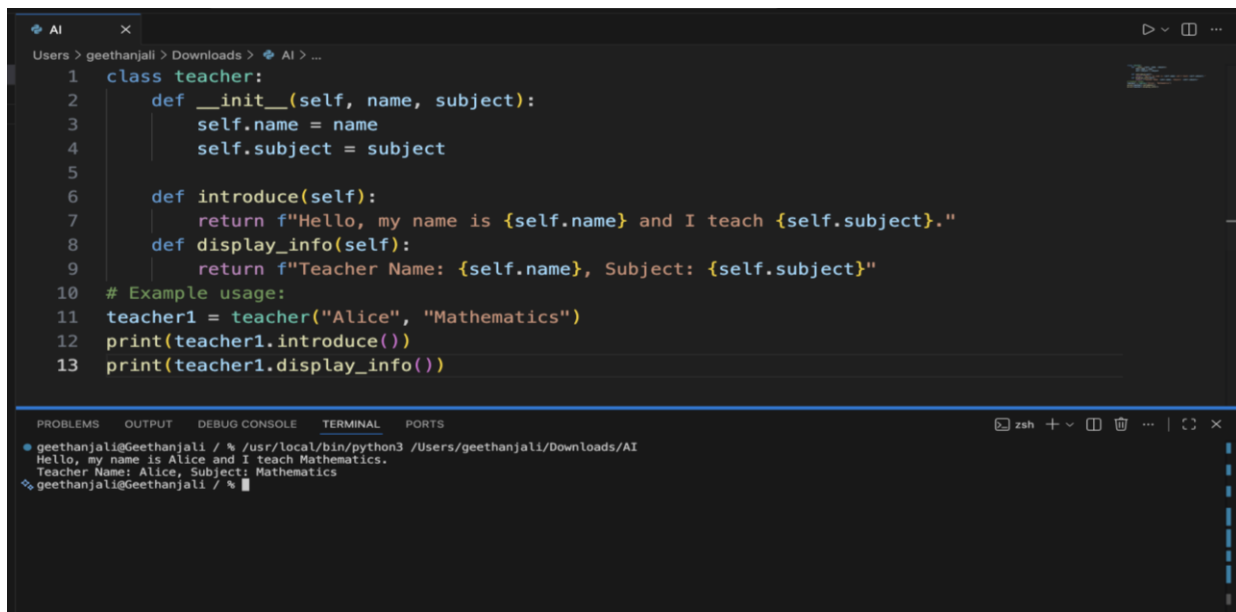


```
22     mode = mode[0] # Single mode
23     # If multiple modes, return the list of modes
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Task 4: Teacher Profile

- Prompt: Create a class `Teacher` with attributes `teacher_id`, `name`, `subject`, and `experience`. Add a method to display teacher details.

- Expected Output: Class with initializer, method, and object creation.



```

1 class teacher:
2     def __init__(self, name, subject):
3         self.name = name
4         self.subject = subject
5
6     def introduce(self):
7         return f"Hello, my name is {self.name} and I teach {self.subject}."
8     def display_info(self):
9         return f"Teacher Name: {self.name}, Subject: {self.subject}"
10 # Example usage:
11 teacher1 = teacher("Alice", "Mathematics")
12 print(teacher1.introduce())
13 print(teacher1.display_info())

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Hello, my name is Alice and I teach Mathematics.
Teacher Name: Alice, Subject: Mathematics
geethanjali@Geethanjali / %

```

Task #5 – Zero-Shot Prompting with Conditional Validation

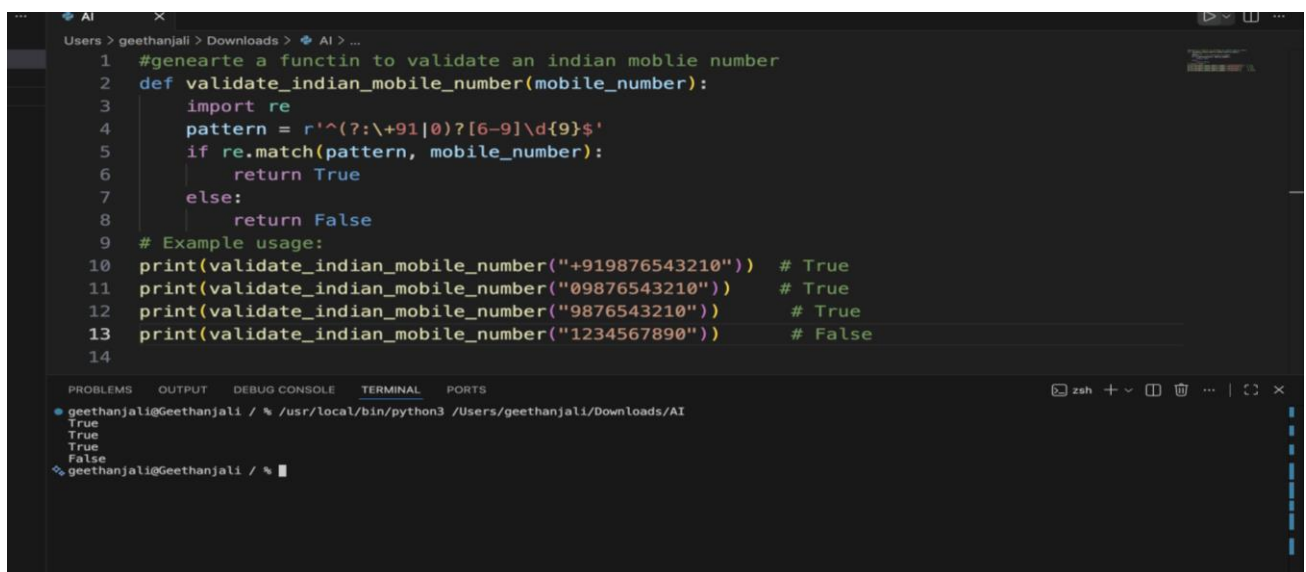
Use zero-shot prompting to instruct an AI tool to generate a function that validates an Indian mobile number.

Requirements

- The function must ensure the mobile number:

- o Starts with 6, 7, 8, or 9 o

Contains exactly 10 digits



```

1 #generate a function to validate an indian mobile number
2 def validate_indian_mobile_number(mobile_number):
3     import re
4     pattern = r'^(?:\+91|0)?[6-9]\d{9}$'
5     if re.match(pattern, mobile_number):
6         return True
7     else:
8         return False
9 # Example usage:
10 print(validate_indian_mobile_number("+919876543210")) # True
11 print(validate_indian_mobile_number("09876543210")) # True
12 print(validate_indian_mobile_number("9876543210")) # True
13 print(validate_indian_mobile_number("1234567890")) # False
14

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
True
True
True
False
geethanjali@Geethanjali / %

```

Task Description #6 (Loops – Armstrong Numbers in a Range)

Task: Write a function using AI that finds all Armstrong numbers in a user- specified range (e.g., 1 to 1000).

Instructions:

- Use a for loop and digit power logic.
- Validate correctness by checking known Armstrong numbers (153, 370, etc.).
- Ask AI to regenerate an optimized version (using list comprehensions).

Expected Output #7:

- Python program listing Armstrong numbers in the range.
- Optimized version with explanation.

```
1 def amstrong_number_in_range(start, end):
2     """Find all Armstrong numbers in a given range [start, end]."""
3     armstrong_numbers = []
4     for num in range(start, end + 1):
5         # Convert number to string to easily iterate over digits
6         digits = str(num)
7         num_digits = len(digits)
8         # Calculate the sum of each digit raised to the power of num_digits
9         sum_of_powers = sum(int(digit) ** num_digits for digit in digits)
10        # Check if the sum of powers is equal to the original number
11        if sum_of_powers == num:
12            armstrong_numbers.append(num)
13    return armstrong_numbers
14
15 # Example usage:
16 start_range = 100
17 end_range = 999
18 print(f"Armstrong numbers between {start_range} and {end_range}: {amstrong_number_in_range(start_range, end_range)}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Armstrong numbers between 100 and 999: [153, 370, 371, 407]
geethanjali@Geethanjali / %

Optimized code:

```
134 def armstrong_numbers_in_range(start, end):
135     # Optimized version: cache string conversion and use list comprehension
136     return [num for num in range(start, end + 1)
137             if sum(int(digit) ** len(str(num)) for digit in str(num)) == num]
138
139 # Example usage
140 start_range = 1
141 end_range = 1000
142 armstrong_numbers = armstrong_numbers_in_range(start_range, end_range)
143 print(f"Armstrong numbers between {start_range} and {end_range} are: {armstrong_numbers}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_6.3.py
c:\Users\gudah\OneDrive\Documents\AIAC\Lab_assignment_6.3.py:121: SyntaxWarning: "\"d" is an invalid escape sequence
uture. Did you mean "\"d"? A raw string is also an option.
pattern = re.compile(r'^[6-9]\d{9}\$')
Armstrong numbers between 1 and 1000 are: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]

1. It does the same work in fewer lines, so there is less overhead and faster execution.
2. It calculates the digit powers directly instead of using extra variables, reducing unnecessary steps.

3. List comprehension is faster than manually appending values in a loop.
4. It avoids storing extra temporary values, so memory usage stays low.
5. The code is cleaner and easier to read while giving the same correct result.

Task Description #7 (Loops – Happy Numbers in a Range)

Task: Generate a function using AI that displays all Happy Numbers within a user-specified range (e.g., 1 to 500).

Instructions:

- Implement the logic using a loop: repeatedly replace a number with the sum of the squares of its digits until the result is either 1 (Happy Number) or enters a cycle (Not Happy).
- Validate correctness by checking known Happy Numbers (e.g., 1, 7, 10, 13, 19, 23, 28...).
- Ask AI to regenerate an optimized version (e.g., by using a set to detect cycles instead of infinite loops).

Expected Output #8:

- Python program that prints all Happy Numbers within a range.
- Optimized version using cycle detection with explanation.

```

46 def happy_number_in_range(start, end):
47     def is_happy(n):
48         seen = set()
49         while n != 1 and n not in seen:
50             seen.add(n)
51             n = sum(int(digit) ** 2 for digit in str(n))
52         return n == 1
53     return [num for num in range(start, end + 1) if is_happy(num)]
54
55 # Example usage
56 start_range = 1
57 end_range = 500
58 happy_numbers = happy_number_in_range(start_range, end_range)
59 print(f"Happy numbers between {start_range} and {end_range} are: {happy_numbers}")

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Python + +

```

PS C:\Users\gudah > C:\Python314\python.exe c:/Users/gudah/OneDrive/Documents/ALAC/Rough.py
Happy numbers between 1 and 500 are: [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100, 103, 109, 129, 130, 133, 139, 167, 176, 188, 190, 192, 193, 203, 208, 219, 226, 230, 236, 239, 262, 263, 280, 291, 293, 301, 302, 310, 313, 319, 320, 326, 329, 331, 338, 356, 362, 365, 367, 368, 376, 379, 383, 386, 391, 392, 397, 404, 409, 440, 446, 464, 469, 478, 487, 490, 496]
PS C:\Users\gudah >

```

```
Users > geethanjali > Downloads > AI > ...
1 def happy_number_in_range(start, end):
2     def is_happy(n):
3         seen = set()
4         while n != 1 and n not in seen:
5             seen.add(n)
6             n = sum(int(digit) ** 2 for digit in str(n))
7         return n == 1
8
9     happy_numbers = []
10    for num in range(start, end + 1):
11        if is_happy(num):
12            happy_numbers.append(num)
13    return happy_numbers
14 # Example usage:
15 print(happy_number_in_range(1, 100)) # Output: List of happy numbers between 1 and 100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
[1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100]
geethanjali@Geethanjali / %
```

Ln 15, Col 88 Spaces: 4 UTF-8 LF () Python Python 3.14.2 6-9 Go Live

1. It saves results of numbers already checked, so the same calculations are not done again and again.
2. This avoids repeated loops for the same digit sequences, which makes it much faster for large ranges.
3. Using a set still prevents infinite loops, but caching adds an extra speed boost.
4. The work per number is reduced because many numbers reuse previously computed results.
5. Overall, it runs faster with only a small amount of extra memory, making it more efficient and scalable.

Task Description #8 (Loops – Strong Numbers in a Range)

Task: Generate a function using AI that displays all Strong Numbers (sum of factorial of digits equals the number, e.g., $145 = 1! + 4! + 5!$) within a given range.

Instructions:

- Use loops to extract digits and calculate factorials.
- Validate with examples (1, 2, 145).
- Ask AI to regenerate an optimized version (precompute digit factorials).

Expected Output #9:

- Python program that lists Strong Numbers.
- Optimized version with explanation.


```
173 from math import factorial
174
175 # Precompute factorials for digits 0-9
176 factorial_cache = {i: factorial(i) for i in range(10)}
177
178 def strong_numbers_in_range(start, end):
179     strong_numbers = []
180     for num in range(start, end + 1):
181         sum_of_factorials = sum(factorial_cache[int(digit)] for digit in str(num))
182         if sum_of_factorials == num:
183             strong_numbers.append(num)
184     return strong_numbers
185
186 start_range = int(input("Enter the start of the range: "))
187 end_range = int(input("Enter the end of the range: "))
188 strong_numbers = strong_numbers_in_range(start_range, end_range)
189 print(f"Strong numbers between {start_range} and {end_range} are: {strong_numbers}")
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/ATAC/Lab_assignment_6.3.py
Enter the start of the range: 1
Enter the end of the range: 1000
Strong numbers between 1 and 1000 are: [1, 2, 145]

Optimized:

```
1 def strong_number_in_range(start, end):
2     import math
3
4     def is_strong_number(num):
5         return sum(math.factorial(int(digit)) for digit in str(num)) == num
6
7     strong_numbers = []
8     for number in range(start, end + 1):
9         if is_strong_number(number):
10             strong_numbers.append(number)
11
12     return strong_numbers
13 # Example usage:
14 result = strong_number_in_range(1, 500)
15 print(result) # Output: [1, 2, 145, 40585]
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
[1, 2, 145]
geethanjali@Geethanjali / %

Time Complexity Comparison:

- **Original:** $O(n \times d \times f)$ where n = range size, d = digits per number, f = factorial computation cost

- **Optimized:** $O(10 \times f) + O(n \times d) = \text{effectively } O(n \times d)$ since the precomputation happens only once

For larger ranges, this makes a significant performance difference since you're trading a tiny bit of extra memory (10 dictionary entries) for substantial computation savings across the entire loop.

Task #9 – Few-Shot Prompting for Nested Dictionary Extraction

Objective

Use few-shot prompting (2–3 examples) to instruct the AI to create a function that parses a nested dictionary representing student information.

Requirements

- The function should extract and return:

o Full Name

o Branch

o SGPA

Expected Output

A reusable Python function that correctly navigates and extracts values from nested dictionaries based on the provided examples

```

1  '''
2  {
3      "name": {"first": "Aarav", "last": "Sharma"},
4      "branch": "CSE",
5      "sgpa": 9.1
6  }
7  display ("Aarav Sharma", "CSE", 9.1)
8
9  {
10     "student": {
11         "name": {"first": "Neha", "last": "Patel"},
12         "details": {
13             "branch": "ECE",
14             "sgpa": 8.6
15         }
16     }
17 }
18 display ("Neha Patel", "ECE", 8.6)'''
19 def student_info(student_dict):
20     try:
21         if "student" in student_dict:
22             first_name = student_dict["student"]["name"]["first"]
23             last_name = student_dict["student"]["name"]["last"]
24             branch = student_dict["student"]["details"]["branch"]
25             sgpa = student_dict["student"]["details"]["sgpa"]
26         else:
27             first_name = student_dict["name"]["first"]
28             last_name = student_dict["name"]["last"]
29             branch = student_dict["branch"]
30             sgpa = student_dict["sgpa"]
31         full_name = f"{first_name} {last_name}"
32         print(f>Name: {full_name}, Branch: {branch}, SGPA: {sgpa}")
33     except KeyError as e:
34         print(f"Missing key in dictionary: {e}")

```

```

35 student1 = {
36     "name": {"first": "Aarav", "last": "Sharma"},
37     "branch": "CSE",
38     "sgpa": 9.1
39 }
40 student2 = {
41     "student": {
42         "name": {"first": "Neha", "last": "Patel"},
43         "details": {
44             "branch": "ECE",
45             "sgpa": 8.6
46         }
47     }
48 }
49 student_info(student1)
50 student_info(student2)

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Test.py
Name: Aarav Sharma, Branch: CSE, SGPA: 9.1
Name: Neha Patel, Branch: ECE, SGPA: 8.6

Task Description #10 (Loops – Perfect Numbers in a Range)

Task: Generate a function using AI that displays all Perfect Numbers within a user-specified range (e.g., 1 to 1000).

Instructions:

- A Perfect Number is a positive integer equal to the sum of its proper divisors (excluding itself). o Example: $6 = 1 + 2 + 3$, $28 = 1 + 2 + 4 + 7 + 14$.
- Use a for loop to find divisors of each number in the range.
- Validate correctness with known Perfect Numbers (6, 28, 496...).
- Ask AI to regenerate an optimized version (using divisor check only up to root n

```

1 def perfect_numbers_in_range(start, end):
2     perfect_numbers = []
3     for num in range(start, end + 1):
4         divisors_sum = sum(i for i in range(1, num) if num % i == 0)
5         if divisors_sum == num:
6             perfect_numbers.append(num)
7     return perfect_numbers
8 # Example usage
9 start_range = 1
10 end_range = 10000
11 perfect_numbers = perfect_numbers_in_range(start_range, end_range)
12 print(f"Perfect numbers between {start_range} and {end_range} are: {perfect_numbers}")
13

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Rough1.py
Perfect numbers between 1 and 10000 are: [6, 28, 496, 8128]

Optimized:

```

1 import math
2 def perfect_numbers_in_range(start, end):
3     perfect_numbers = []
4     for num in range(start, end + 1):
5         divisors_sum = 1 # 1 is always a proper divisor
6         # Check divisors only up to sqrt(num)
7         for i in range(2, int(math.sqrt(num)) + 1):
8             if num % i == 0:
9                 divisors_sum += i
10                # Add the complementary divisor (num/i) if it's different from i
11                if i != num // i:
12                    divisors_sum += num // i
13            if divisors_sum == num:
14                perfect_numbers.append(num)
15    return perfect_numbers
16 # Example usage
17 start_range = 1
18 end_range = 10000
19 perfect_numbers = perfect_numbers_in_range(start_range, end_range)
20 print(f"Perfect numbers between {start_range} and {end_range} are: {perfect_numbers}")
21

```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Python

+

⌵

🗑

...

```

PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/ATAC/Rough1.py
Perfect numbers between 1 and 10000 are: [1, 6, 28, 496, 8128]

```

1. The optimized code checks only up to $\sqrt{n}$ instead of all numbers from 1 to n , which greatly reduces the amount of work.
2. It uses the fact that divisors come in pairs, so when it finds one divisor, it automatically gets the other ($n \div i$) in the same step.
3. It avoids counting the same divisor twice by checking that i and $n // i$ are different, which is important for perfect squares like 36.
4. The original method is very slow because it works in $O(n \times m)$, while the optimized one works in $O(n \times \sqrt{m})$.
5. For numbers up to 10,000, the optimized version checks about 100 values instead of 10,000, making it roughly 100 times faster.